

S32K344 도메인 담당자 중점 스터디 상세계획

기준 보드: S32K344-WB / S32K344

개발 환경: S32 Design Studio 3.6.x + S32K3 RTD

대상 역할: Central / Domain Controller 담당자

방향: S32K144 에서 이미 수행한 GPIO, Timer, Interrupt, ADC, PWM, UART, CAN 주변장치 기초는 최대한 압축하고,

실제 프로젝트에서 도메인 제어가 담당하는 상태 취합 → 요청 검증 → 판단/중재 → 명령 분배 → 결과/Fault 통합을 중심으로 학습한다.

▼ 1. 스터디 방향

기존 S32K344 스터디는 MCU 전체 기능을 폭넓게 다루는 14 일 과정이었다.

하지만 현재 프로젝트에서 담당하는 S32K344 는 개별 액추에이터를 직접 구동하는 ECU 가 아니라 중앙 도메인 제어기이므로 학습 우선순위를 다음과 같이 변경한다.

기존 S32K344 전체 학습
GPIO / Timer / Interrupt / PWM / ADC / DMA / UART / SPI
CAN / CAN FD / FreeRTOS / Ethernet / Safety / HSE

↓
담당 역할 기준 압축

↓
RTD 구조와 S32K3 차이

↓
FreeRTOS / 주기 제어

↓
ECU Interface

↓
Request / State / Event / Fault

↓
요청 검증 및 중재

↓
상태 최신성 / Heartbeat / Timeout

↓
Smart Access / Climate / Ambient 등 중앙 판단

↓
통합 / Fault / HMI 결과 관리

▼ 2. SR 기준 도메인 제어기 핵심 역할

Central Controller 가 중점적으로 수행할 부분은 다음과 같다.

1. 기능별 ECU 상태 취합

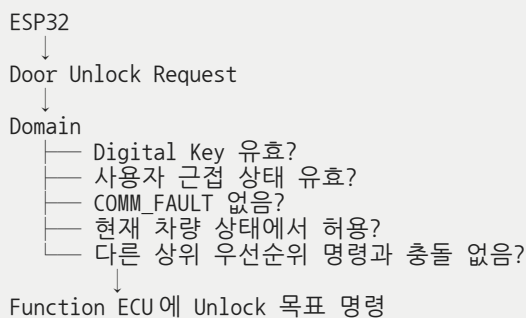
대상 예:

- Power Window Controller
- Function Controller
- Sensing Controller
- Vision 결과
- ESP32 / 모바일 사용자 요청
- VSS 상태 또는 Event

도메인에서는 각 장치를 직접 제어하기보다 각 ECU 가 제공하는 상태와 결과를 저장하고 관리한다.

2. 사용자 / 센싱 요청 검증

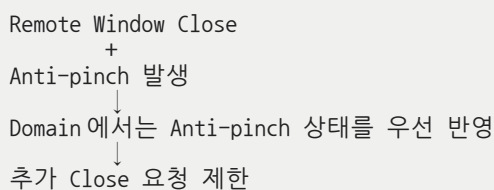
예:



3. 요청 중재 및 기능 허용 판단

동시에 여러 요청이 발생했을 때 어떤 요청을 허용할지 판단한다.

예:



실제 모터 보호 동작은 Window ECU 가 수행하더라도 도메인은 이후 요청을 허용할지 판단할 수 있어야 한다.

4. 자동 기능 연계 판단

대표 기능:

- Smart Access
- Predictive Climate
- Auto Re-lock

- 자동 환기
- Ambient Lighting
- 차량 상태 기반 경고 / HMI 표시

5. Fault 및 상태 최신성 관리

도메인에서는 단순히 값만 저장하면 안 된다.

```
ECU State
+
Last Update Time
+
Valid
+
Fault
+
Communication State
```

를 함께 관리해야 한다.

예:

```
Window Position = 50 %
```

라는 값이 있더라도 2 초 동안 새로운 상태가 수신되지 않았다면

```
현재 Window Position = 50 %
```

라고 신뢰해서는 안 된다.

따라서 다음 개념이 중요하다.

- Heartbeat
- Update Period
- Timeout
- Data Freshness
- COMM_FAULT
- SENSOR_FAULT
- FUNCTION_FAULT
- Recovery 조건

▼ 3. 기존 S32K144 학습 내용 압축 기준

이미 S32K144 에서 학습한 내용은 다음처럼 처리한다.

항목	이번 S32K344 스터디
GPIO	S32K3 SIUL2 차이만 확인
Clock	Clock Tree 위치와 Peripheral Clock 확인만
Timer	FreeRTOS Tick/주기 Task 이해에 필요한 정도
Interrupt	NVIC + ISR → Task 전달만 복습
PWM	제외
ADC	제외
SPI	제외
일반 UART	ESP32 Interface 이해용으로만 간단히
CAN 기본	송수신 구현 반복하지 않음
CAN FD Bit Timing	통신 담당자 영역. 개념만 이해
DMA	선택
Ethernet	선택
HSE	선택
Safety	Watchdog/Fault 관점만 간단히
FreeRTOS	중점
Request / State 구조	최중점
ECU 상태 취합	최중점
Timeout / Heartbeat	최중점
통합 판단 로직	최중점

▼ 4. 전체 일정

기존 14 일 → 10 일로 압축한다.

권장 시간:

1 일 2 시간 전후
총 약 18~22 시간

시간이 매우 부족하면 Day 1~2 를 하루로 합쳐 9 일 과정으로 진행해도 된다.

Day	주제	중요도
1	S32K3 RTD / 프로젝트 구조 + S32K144 차이 압축	★★★
2	GPIO / Clock / Timer / Interrupt 초 압축 복습	★★

Day	주제	중요도
3	FreeRTOS 기반 Domain 실행 구조	★★★★★
4	Domain SW Architecture + Request / State 모델	★★★★★
5	ECU Interface + 상태 수신 구조	★★★★★
6	Request 검증 / 중재 / 명령 분배	★★★★★
7	Heartbeat / Timeout / Fault / Data Freshness	★★★★★
8	Smart Access / Climate / Ambient 통합 판단	★★★★★
9	ESP32/HMI 연동 + Request 결과 관리	★★★★★
10	Domain 통합 미니 프로젝트 + 검증	★★★★★

▼ Day 1. S32K3 RTD / 프로젝트 구조

목표

S32K144 와 다른 S32K344 개발 흐름만 빠르게 익힌다.

깊게 공부할 필요가 없는 주변장치 실습 대신 다음 흐름을 정확히 이해한다.

```

.mex / Configuration Tool
    ↓
Generated Configuration
    ↓
RTD Driver
    ↓
Application Code

```

확인할 것

- `src/`
- Generated code
- RTD Driver
- Startup code
- Linker script
- `.mex`
- `*_Ip_*` 계층
- Config Tool 에서 만든 설정이 코드에 어떻게 연결되는지

실습

기본 프로젝트를 Build / Debug 하고 `main()`까지 진입한다.

그리고 다음 정도만 실행한다.

```
Clock Init  
Peripheral Init  
RTOS 또는 Main 진입
```

제출

S32K344 프로젝트 초기화 흐름

을 10~15 줄 이내로 정리한다.

예상 시간

1.5~2 시간

▼ Day 2. S32K144 내용 초압축 복습

이 Day 의 목적은 새로운 주변장치 기능을 배우는 것이 아니다.

S32K144 에서 했던 작업이 S32K344 에서는 어디가 달라지는지만 확인한다.

Part A. GPIO / SIUL2

확인:

```
S32K144  
PORT + GPIO  
  
S32K344  
SIUL2 MSCR / IMCR + Dio / Port
```

실습:

Button → LED Toggle

한 번만 구현한다.

Part B. Clock

다음 경로만 확인한다.

```
Clock Source
↓
PLL / Divider
↓
Core Clock
↓
Peripheral Clock
```

계산 연습을 반복하지 않는다.

Part C. Timer / Interrupt

100 ms Timer 또는 RTOS Tick 중 하나를 이용하여

100 ms 마다 Counter 증가

만 확인한다.

중요한 부분:

- ISR에서는 최소 작업
- ISR → Event/Flag/Queue
- 실제 판단은 Task에서 수행

생략

이번 과정에서는 다음 실습을 별도로 하지 않는다.

- PWM 밝기 제어
- ADC → PWM
- SPI Device
- DMA Memory Copy

예상 시간

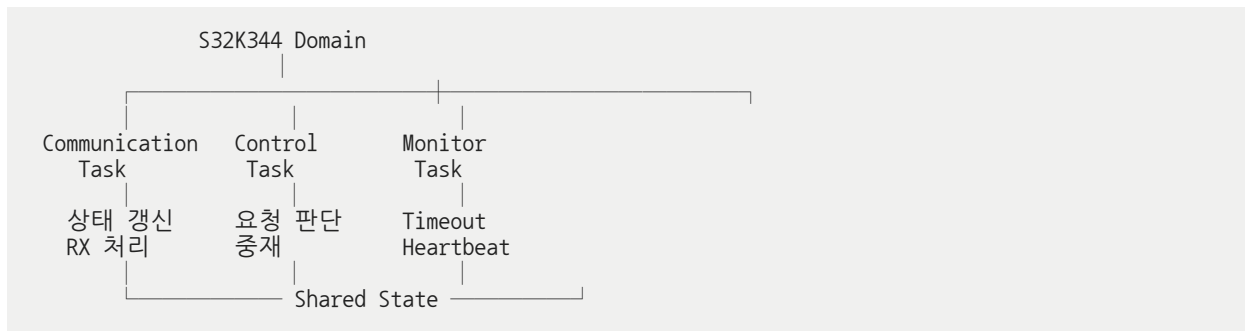
1.5~2 시간

▼ Day 3. FreeRTOS 기반 Domain 실행 구조

목표

이번 스터디에서 가장 중요한 Day 중 하나이다.

도메인 컨트롤러를 다음과 같이 Task 기반으로 분리하는 방식을 이해한다.



학습

- Task
- Priority
- Tick
- `vTaskDelay()`
- `vTaskDelayUntil()`
- Queue
- Event
- Semaphore
- Mutex
- ISR → Task

실습

다음 Task 를 만든다.

Task_Input	10 ms
Task_Control	20 ms
Task_Monitor	100 ms
Task_Status	1000 ms

아직 실제 CAN 데이터를 사용하지 않아도 된다.

가짜 상태값을 생성하여 Task 간 데이터가 전달되도록 한다.

핵심 질문

- 어떤 작업을 주기 Task 로 만들 것인가?
- 어떤 작업을 Event 기반으로 만들 것인가?
- Control Task 주기는 얼마가 적당한가?
- Communication Task 와 Control Task 를 분리해야 하는 이유는?
- Shared State 를 동시에 접근하면 어떤 문제가 생기는가?

예상 시간

2~2.5 시간

▼ Day 4. Domain SW Architecture + Request / State 모델

목표

프로젝트 실제 코드 구조의 기반을 만든다.

도메인 코드에서는 Request 와 State 를 분리한다.

Request = 무엇을 해달라는 요구

State = 현재 실제 상태

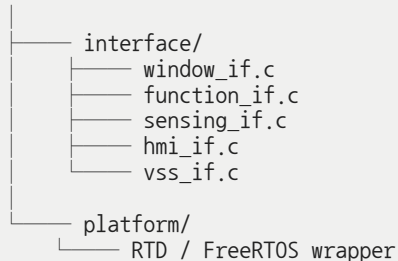
예:

```
typedef enum
{
    REQ_NONE,
    REQ_WINDOW_OPEN,
    REQ_WINDOW_CLOSE,
    REQ_WINDOW_STOP
} WindowRequest;

typedef struct
{
    uint8_t position;
    bool moving;
    bool anti_pinch;
    bool valid;
    uint32_t last_update_ms;
} WindowState;
```

추천 구조

```
Domain/
├── app/
│   ├── domain_main.c
│   └── domain_task.c
├── model/
│   ├── vehicle_state.h
│   ├── vehicle_request.h
│   ├── vehicle_event.h
│   └── vehicle_fault.h
└── control/
    ├── request_manager.c
    ├── state_manager.c
    ├── domain_control.c
    └── fault_manager.c
```



실습

최소 다음 구조체를 만든다.

```

VehicleState
WindowState
FunctionState
SensingState
HmiState

VehicleRequest
WindowRequest
DoorRequest
ClimateRequest
LightingRequest

FaultState
  
```

핵심 목표

Peripheral Driver 보다 Application Data Model 을 먼저 안정적으로 설계하는 습관을 만든다.

예상 시간

2~2.5 시간

▼ Day 5. ECU Interface + 상태 취합

목표

통신 자체를 깊게 구현하는 것이 아니라

```

통신 담당자가 전달한 데이터
↓
Domain Interface Layer
↓
Domain State
  
```

구조를 만든다.

역할 분리

통신 담당

예:

```
CAN RX
ID Parsing
Payload Decode
CRC / Sequence
Driver Callback
```

Domain 담당

예:

```
WindowState 업데이트
FunctionState 업데이트
SensingState 업데이트
last_update 갱신
valid 판정
Control Logic 에서 사용
```

실습

다음 함수를 가정한다.

```
void Domain_UpdateWindowState(const WindowState *state);
void Domain_UpdateFunctionState(const FunctionState *state);
void Domain_UpdateSensingState(const SensingState *state);
```

통신 대신 Dummy 데이터를 넣어도 된다.

상태 관리 구조

각 State 에 최소 다음 정보를 둔다.

```
Data
Valid
Fault
Last Update
Sequence
```

예:

```
typedef struct
{
    uint8_t position;

    bool moving;
    bool anti_pinch;
```

```

bool valid;
bool comm_fault;

uint8_t sequence;
uint32_t last_update_ms;
} WindowState;

```

예상 시간

2 시간

▼ Day 6. Request 검증 / 중재 / 명령 분배

목표

도메인 컨트롤러의 핵심 기능을 구현한다.

단순히

HMI 요청 → ECU 전달

이 아니라

```

HMI 요청
↓
Request Validation
↓
Current State 확인
↓
Fault 확인
↓
Priority / Conflict 확인
↓
Allow / Reject
↓
Target Command 생성

```

구조를 구현한다.

실습 1. Window 요청 검증

예:

```

CLOSE Request
↓
COMM_FAULT ?
├── YES → Reject
└── NO
    ↓
    Anti-pinch 상태 ?
    ├── YES → Reject
    └── NO
        ↓
        State Valid ?
        └── NO → Reject

```

↓ YES
CLOSE Command

실습 2. Door Unlock

Unlock Request
↓
Digital Key Valid ?
↓
Proximity Valid ?
↓
Vehicle State 허용 ?
↓
Function ECU Command

결과 모델

요청 결과도 구조화한다.

```
typedef enum  
{  
    RESULT_PENDING,  
    RESULT_ACCEPTED,  
    RESULT_REJECTED,  
    RESULT_COMPLETED,  
    RESULT_FAILED  
} RequestResult;
```

거부 사유도 정의한다.

```
INVALID_STATE  
COMM_FAULT  
SENSOR_INVALID  
AUTH_FAILED  
CONFLICT  
TIMEOUT
```

예상 시간

2~2.5 시간

▼ Day 7. Heartbeat / Timeout / Fault / Data Freshness

목표

실제 도메인 개발에서 매우 중요한 부분이다.

Heartbeat

각 ECU 에서 일정 주기로 상태를 전달한다고 가정한다.

예:

```
Window ECU    100 ms
Function ECU   100 ms
Sensing ECU    100 ms
```

실제 주기는 팀 Interface Specification 에서 결정한다.

Timeout

예:

```
Expected Update = 100 ms
Timeout          = 500 ms
```

이면

```
if ((now - state.last_update_ms) > TIMEOUT_MS)
{
    state.valid = false;
    state.comm_fault = true;
}
```

형태로 관리할 수 있다.

중요한 원칙

잘못된 방법

```
마지막으로 받은 값 계속 사용
```

권장

```
value
+
timestamp
+
valid
+
fault
```

실습

임의로 Window ECU 상태 갱신을 중단한다.

0~3 초 : 정상 상태 수신
3 초 이후 : 수신 중단

Domain 에서

NORMAL
↓
STALE
↓
COMM_FAULT

로 전환되는 것을 확인한다.

다시 상태가 들어오면 Recovery 조건에 따라 정상화한다.

Fault 격리

Window ECU 가 Fault 라고 해서

Lighting
Climate
Door Lock

까지 모두 정지시키지 않는다.

기능 단위 Fault 영향을 정의한다.

예상 시간

2~2.5 시간

▼ Day 8. SR 기능 기반 중앙 판단

목표

실제 SR 기능을 Domain Logic 으로 옮긴다.

Part A. Smart Access

입력:

Digital Key Valid
Proximity

Door State
Vehicle State
Communication State

판단:

LOCK
UNLOCK
REJECT

Part B. Predictive Climate

입력:

Temperature
Humidity
Occupant
Sensor Valid
Vehicle State

출력 예:

CLIMATE_OFF
FAN_LOW
FAN_MID
FAN_HIGH

실제 Threshold 는 추후 SysRS / 설계값에 맞춘다.

Part C. Ambient Lighting

입력:

Illuminance
Access State
Climate State
Warning
Fault
Emergency

중앙에서는 개별 LED PWM 제어 대신

NORMAL
WELCOME
CLIMATE_ACTIVE
WARNING
FAULT
EMERGENCY

같은 의미 상태(Semantic State)를 생성하는 방식으로 설계한다.

실제 LED 출력은 Function Controller 가 담당한다.

Part D. 자동 환기

예:

```
Temperature / Humidity
Occupant State
Window State
Fault
```

을 확인해 자동 환기 허용 여부를 판단한다.

실습

최소 3 개의 Scenario 를 만든다.

```
Scenario 1
정상 Smart Access

Scenario 2
Sensor Fault 중 Climate Request

Scenario 3
Window COMM_FAULT 중 Auto Ventilation Request
```

각 Scenario 에 대해

```
Input
Decision
Output Command
Reason
```

을 출력한다.

예상 시간

2~3 시간

▼ Day 9. ESP32 / HMI Request-Result 연동

목표

HMI 가 단순 Command Sender 가 아니라

```
Request
↓
Domain
↓
```

```

Accepted / Rejected
  ↓
실제 ECU 처리
  ↓
Completed / Failed
  ↓
HMI 상태 표시

```

구조로 동작하도록 설계한다.

Request 흐름

```

Mobile App
  ↓
ESP32
  ↓
S32K344 Domain
  ↓
Request Validation
  ↓
Target ECU
  ↓
State / Result
  ↓
S32K344 Domain
  ↓
ESP32
  ↓
Mobile App

```

구현할 데이터

Request

```

Request ID
Target
Command
Parameter
Timestamp

```

Result

```

Request ID
Result
Reject / Failure Reason
Current State

```

실습

Dummy ESP32 입력을 사용해도 된다.

REQ 12 DOOR_UNLOCK

입력 후

REQ 12 ACCEPTED

또는

REQ 12 REJECTED AUTH_FAILED

가 반환되도록 한다.

이후 ECU 상태까지 반영되면

REQ 12 COMPLETED

를 반환한다.

예상 시간

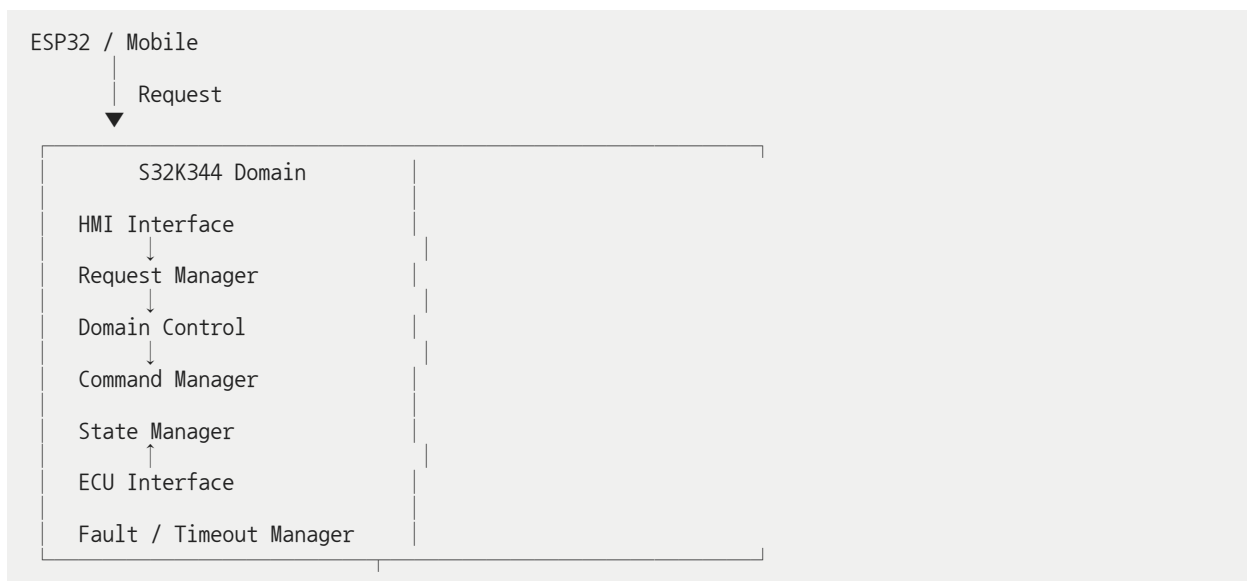
2 시간

▼ Day 10. Domain Controller 통합 미니 프로젝트

목표

지금까지 만든 요소를 하나로 통합한다.

전체 구조





FreeRTOS 예시

Task_CommRx	Event Driven / 5~10 ms
Task_Control	20 ms
Task_Monitor	100 ms
Task_HmiUpdate	100 ms
Task_Log	1000 ms

주기는 프로젝트 Interface 가 확정되면 조정한다.

통합 Scenario

Scenario 1 — 정상 Unlock

```

Digital Key Valid
+
Near
+
No Fault
↓
Unlock Accepted
↓
Function ECU Command
↓
Door Unlocked
↓
HMI Completed
  
```

Scenario 2 — ECU Timeout

```

Function ECU Update 중단
↓
Timeout
↓
COMM_FAULT
↓
Door Request Reject
↓
HMI Fault 표시
  
```

Scenario 3 — Sensor Fault

```

Temperature Invalid
↓
SENSOR_FAULT
↓
Predictive Climate 제한
↓
다른 독립 기능은 정상 유지
  
```

Scenario 4 — Window Safety State

```
Anti-pinch Active
  ↓
Close Request Reject
  ↓
Window 상태 / 경고 HMI 반영
```

▼ 5. 최종 산출물

스터디가 끝났을 때 단순 Peripheral 예제보다 다음 문서/코드가 남는 것이 중요하다.

1. Domain SW Architecture

```
domain_architecture.md
```

내용:

- Layer
- Module
- Task
- Data Flow

2. ECU Interface 정의

```
ecu_interface.md
```

대상:

- Window
- Function
- Sensing
- ESP32
- VSS

3. Vehicle State 정의

```
vehicle_state.h
```

4. Vehicle Request 정의

```
vehicle_request.h
```

5. Fault 정의

```
vehicle_fault.h
```

6. FreeRTOS Task 표

예:

Task	역할	Trigger / 주기
CommRx	수신 데이터 Domain State 반영	Event
Control	요청 검증 / 중앙 판단	20 ms
Monitor	Timeout / Heartbeat / Fault	100 ms
HMI Update	상태 및 결과 전달	100 ms
Log	Debug 상태 출력	1000 ms

7. Scenario Test

최소 다음을 시험한다.

- 정상 Request
- Invalid Request
- COMM_FAULT
- SENSOR_FAULT
- Request Conflict
- Timeout
- Recovery
- ECU 한 개 Fault 시 다른 기능 독립 동작

▼ 6. 통신 담당자와의 경계

현재 통신을 다른 담당자가 맡는다면 다음까지는 깊게 공부하지 않아도 된다.

통신 담당자 중심

- FlexCAN Instance 설정
- CAN Bit Timing
- CAN FD BRS
- Transceiver
- Termination
- Message Buffer 상세 설정
- CAN ISR / Driver 상세 구현
- Payload Byte/Bit Packing

Domain 담당자가 반드시 알아야 하는 부분

- 어떤 ECU 에서 어떤 State 가 오는가
- 어떤 Command 를 보내야 하는가
- 상태 갱신 주기는 얼마인가
- Timeout 은 얼마인가
- Sequence 가 필요한가
- Data Valid 조건은 무엇인가
- Fault 일 때 어떤 요청을 막는가
- Request 결과를 어떻게 HMI 에 반환하는가

즉,

CAN 을 어떻게 보내는가

보다

무슨 정보를 언제 보내고,
도메인이 그 정보를 어떻게 판단하는가

가 더 중요하다.

▼ 7. 이번 스터디에서 후순위로 내릴 항목

다음 항목은 담당 범위에 필요해질 때 다시 학습한다.

후순위

- eMIOS PWM 상세
- ADC 상세

- LPSPI 상세
- eDMA 상세
- Ethernet 상세
- CAN FD Bit Timing 계산
- HSE Crypto
- Secure Boot
- Audio Codec / VSS 출력
- 개별 센서 Driver
- 개별 Motor Driver

알아만 둘 것

- Watchdog
- ECC
- Cortex-M7 Cache
- Safety 개념
- HSE 역할

▼ 8. 시간 부족 시 최우선 순위

시간이 정말 부족하면 아래 순서대로 진행한다.

1. Day 4 Request / State Data Model
2. Day 5 ECU Interface
3. Day 6 Request Validation / Arbitration
4. Day 7 Heartbeat / Timeout / Fault
5. Day 3 FreeRTOS Task 구조
6. Day 8 실제 SR 기능 중앙 판단
7. Day 10 통합
8. Day 9 ESP32/HMI 결과 처리
9. Day 1~2 S32K344 Hardware/RTD 복습

즉, 이번 S32K344 학습의 목표를

"S32K344의 모든 Peripheral을 사용할 줄 아는 사람"

으로 잡기보다

"도메인 ECU의 상태와 요청을 설계하고
S32K344 + FreeRTOS 위에서 중앙 판단 구조를 구현할 수 있는 사람"

으로 잡는다.

▼ 9. Repository 권장 구조

```
s32k344-domain-study/  
├── assignments/  
│   ├── day01-rtd/  
│   ├── day02-s32k3-review/  
│   ├── day03-freertos/  
│   ├── day04-domain-model/  
│   ├── day05-ecu-interface/  
│   ├── day06-request-manager/  
│   ├── day07-fault-monitor/  
│   ├── day08-domain-logic/  
│   ├── day09-hmi-interface/  
│   └── day10-integration/  
├── firmware/  
│   └── domain/  
│       ├── app/  
│       ├── model/  
│       ├── control/  
│       ├── interface/  
│       └── platform/  
├── docs/  
│   ├── domain_architecture.md  
│   ├── ecu_interface.md  
│   ├── task_design.md  
│   ├── fault_policy.md  
│   └── scenario_test.md  
└── README.md
```

▼ 10. 최종 목표

이 스터디 이후에는 다음 질문에 답할 수 있어야 한다.

- Power Window ECU 에서 어떤 상태를 받아야 하는가?
- Function ECU 에서 어떤 상태를 받아야 하는가?
- Sensing ECU 데이터는 언제 Invalid 로 판단하는가?
- Request 와 State 는 왜 분리하는가?
- Command 와 Request 는 어떻게 다른가?
- 같은 기능에 두 요청이 동시에 들어오면 어떻게 처리하는가?
- ECU 통신이 끊기면 몇 ms 후 Fault 로 판단하는가?
- Fault 가 발생했을 때 어떤 기능을 제한하고 어떤 기능은 유지하는가?
- FreeRTOS Task 를 어떻게 나눌 것인가?
- ISR 에서 받은 데이터를 Control Task 로 어떻게 전달할 것인가?
- HMI 에는 요청 접수 / 거부 / 성공 / 실패를 어떻게 전달할 것인가?
- Smart Access / Climate / Ambient 판단을 어느 Module 에 둘 것인가?
- 새 ECU 하나가 추가되면 Interface 와 State 구조를 어떻게 확장할 것인가?

이 질문에 코드와 설계 문서로 답할 수 있으면 S32K344 Domain Controller 담당자로 필요한 핵심 스테디는 충분히 수행한 것으로 본다.